LiveRecall: architecture and revision semantics

Team review draft, CS3103 Assignment 4. This report describes peer components assembled for server testing and Leonard's overload policy and experiments. The browser demo and final submission deliverables are not yet complete.

Data path and protocol

The unchanged deterministic caption_source.py connects using SOURCE_HELLO. One L1 source sends CAPTION_UPDATE messages with segment, revision, source_ts, final and text. Segments increase; revision counts are variable. The server parses updates, applies revision state, adds accepted finals to recovery history and publishes accepted updates through the Hub. Unknown fields are omitted from the compact encoded caption. SOURCE_DONE does not become a caption.

Don's CaptionState accepts a previously unseen segment or a strictly newer revision of an unfinished segment. Once final, the segment cannot be overwritten. One wire representation is reused across recipients and recovery history. PR #19, now on `main`, prunes global revision metadata below the oldest retained final and rejects replay of evicted segments. Don and Dylan should review this cross-component behavior before final submission.

Independent delivery and overload

Damian's Hub publishes synchronously to separate client mailboxes, without awaiting sockets in the source loop. Each client has one writer. Unimpaired clients receive every revision. Actual lag activates coalescing: pending provisionals for a segment replace obsolete versions while finals take priority. An overdue ping acknowledgement or full unacknowledged frame window detects a lagging reader even if kernel buffers still accept writes. Catch-up can batch pending finals into one HISTORY frame.

The recorded public campaign used a 4096-byte write limit, inbound frame high-water mark 16, compression off, 512 pending finals and listen backlog 4096. These match the retained controlled comparison. The later H3/H8 fix uses 1024-byte writes, deflate and a bounded idle final flush; its separate evidence and limitations are in docs/congestion-recovery.md. The Hub's existing one-second probe interval remains unchanged. Startup tries to raise the descriptor soft limit to the existing hard limit. Environment settings are validated.

The policy rejects more than 1024 pending updates per client, or more than 512 pending finals. One additional frame may be in flight. Overflow closes code 1013; the client reconnects to recover within bounded history rather than silently losing finals. FIFO comparison mode preserves queue order under the same bounds and bypasses semantic probes, prioritization and batching.

Recovery, browser behavior and integration

Incremental recovery

Dylan's FinalHistory retains finals only, bounded by 90 seconds and 512 items. CLIENT_HELLO carries the last finalized segment/revision and requested history window up to 60 seconds. New clients receive recent finals; returning clients receive only newer finals within the recovery window. HISTORY is oldest first and contains no provisional captions. The server builds the history snapshot and registers the writer synchronously, without an await between them, preventing a live update from slipping through the handoff. Dylan's merged cache reuses identical encoded HISTORY replies during reconnect bursts, bounded to 64 entries and 8 MiB; each new final invalidates the cache.

The cursor reflects finalized state, not a displayed provisional. The server does not replay the current provisional on resume, avoiding a duplicate revision. Damian's connection module retries using jittered exponential backoff, resends CLIENT_HELLO and reports status. A fast empty HISTORY alone does not prove recovery: T4 checks final completeness and duplicates as well as recovery time.

Browser shell

Leonard's shell replaces the row for a segment and distinguishes provisional, final and recovered text. It shows Connected, Reconnecting or Offline and the last-update age. The URL defaults to the page hostname on WebSocket port 8765, with a query override. Remote text uses textContent. Live rows, recall rows and the pending render map each have a 300-entry bound. Semantic processing is immediate; painting uses one requestAnimationFrame callback. Scrolling up pauses following, with a Return to live control. Recall defaults to 30 seconds with 15/30/60 buttons.

Don's ES-module store, Damian's connection manager and Dylan's browser history are merged on `main`. This integration PR adds Leonard's shell from PR #13. The history keeps 90 seconds of finals with a 512-item bound, merges recovered captions without duplication and serves recall locally. A headless Chromium two-tab check exercised live captions, late join, offline/reconnect and recovered rows with the deterministic source. The team must still review the user-facing demo and record its video.

Remaining correctness checks

PR #19 merged bounded server revision state, a 512-entry client-store retention floor and protocol close code 1008 for malformed HELLO cursors. Don and Dylan should review those changes before final submission. A human-reviewed two-tab demo must verify revision replacement, all recall windows, disconnect and incremental recovery without duplicates, late joining and readable status. Peer code assembled for measurement retains its original ownership and review needs.

Public experiments and controlled optimization

Method

The existing Ubuntu 22.04.5 VM runs Python 3.10.12 and websockets 16.1.1. T1–T5 run sequentially in one private user/network namespace with 16-CPU affinity. T1: 20 clients, 20 seconds. T2: 100 clients, 40 seconds. T3: 100 clients, 60 seconds, 10% slow readers at 500 ms. T4: 100 clients, 60 seconds, 20% disconnecting at 15 seconds for 5 seconds. T5: T2 with loopback delay 120 ms plus/minus 30 ms and 3% loss, affecting source and client traffic. A trap removes shaping. Instructor generators, profiles and measurement logic remain unchanged.

Each case restarts the server three times. Results are medians of per-run metrics, not pooled percentiles. Freshness uses the public analyser's 500 ms threshold. experiments/public retains original compressed CSV, per-run JSON, logs, RSS samples and source/settings metadata. Summary JSON also includes normal-reader latency, coverage and recovery where the analyser supplies those fields.

Case	P50 ms	P95 ms	P99 ms	Fresh %	Final %	Fresh final %
T1	1.34	2.15	2.59	100.00	100.00	100.00
T2	4.65	7.85	8.64	100.00	100.00	100.00
T3	4.12	7.60	5592.37	97.07	100.00	94.34
T4	4.62	7.65	8.57	100.00	100.00	98.42
T5	247.99	617.18	853.69	93.29	100.00	89.49
T3 fifo	5.24	2254.10	24049.24	94.65	95.13	90.00
T3 semantic	5.83	12.01	5588.78	97.07	100.00	94.34

T1 and T2 have 100% median revision coverage and final completeness. In T3, normal-reader P95 is 7.03 ms (T2: 7.85 ms); all 90 normal readers receive all 228 revisions each. The analyser's normal_update_coverage_pct field aggregates all clients, including slow ones, despite its name. T3 RSS is about 26–27 MiB per server process across these short runs; this does not prove long-run bounds.

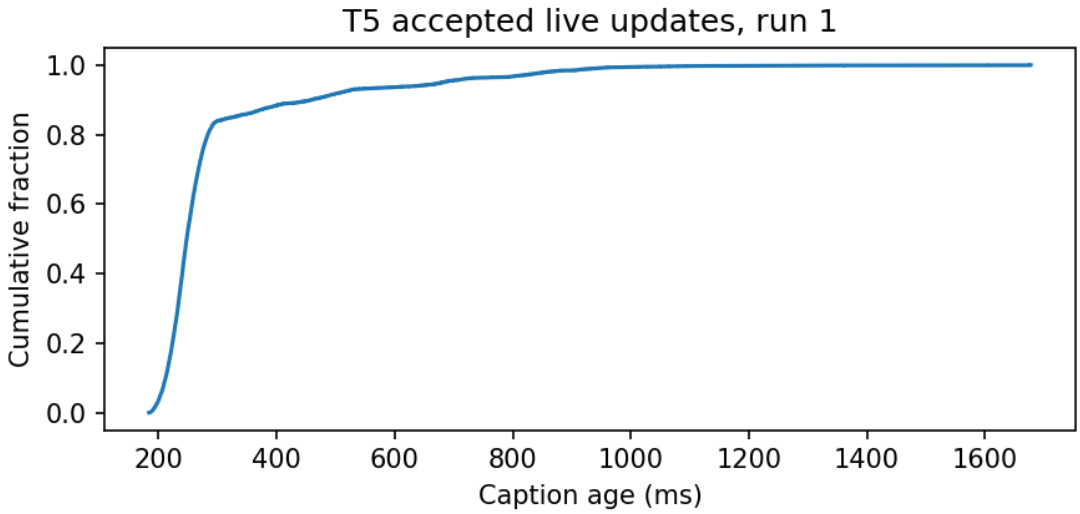
T4 has 100% median final completeness and median recovery P95 of 2.51 ms. No duplicate, stale, invalid-history or connection-error events occur in its three runs. Recovery timing is measured alongside delivered finals, not in isolation.

One before/after comparison

The paired T3 rerun used the same assembled Git revision in one private network namespace with four-CPU affinity. Both modes ran sequentially with 100 clients, 60 seconds, 10% slow readers at 500 ms, three server restarts, seed 3103, 4096-byte writes, queue 16 and no compression. Only `LR_DELIVERY` changed from FIFO to semantic. Metadata records the same revision, CPU affinity, namespace and effective settings except delivery. The separate public campaign used 16 CPUs, so its absolute latencies should not be treated as the before/after comparison.

Median final completeness improved from 95.13% in FIFO to 100.00% in semantic mode; overall P99 fell from 24049.24 ms to 5588.78 ms. Normal-reader P95 rose from 8.46 to 10.40 ms, while slow-reader P95 fell from 28121.03 to 9008.18 ms. Coalescing obsolete provisionals and batching finals help lagging readers converge. This does not prove every final is fresh or establish the best setting for hidden bandwidth tests. The large optional sweep and repeated matrices were removed from the submission.

Interpretation, limitations and future work



The CDF shows accepted live captions in T5 run 1. The preceding table uses all three runs. TCP retransmits lost data and delivers bytes in order: packet loss principally appears as delayed captions and larger tail age. Later bytes can wait for a missing earlier range. Semantic queues replace pending obsolete provisionals, but cannot edit frames already sent to TCP. This plot alone does not identify the retransmission responsible for any particular delay. All three refreshed T5 runs delivered 4,900 of 4,900 finals. One run in the earlier separate campaign missed its last final before test end; changed netem randomness and timing mean the difference cannot be attributed to the retention fix.

Trade-offs and limitations

Consider T3's normal/slow metrics and RSS alongside overall percentiles. Isolating normal readers does not make a slow reader fast. Final priority and batching improve completeness, while recovered captions may be too late to count as fresh live updates. Overflow eviction bounds memory but adds reconnect work, and recovery is limited by history retention. Compression trades bandwidth for CPU/memory; no optimum is claimed. The memory plot retains separate server processes across the three restarts.

These are deterministic local public workloads. Shared VM activity and unfixed netem loss randomness can vary between repeats. The four-CPU comparison had a different CPU partition from the 16-CPU public tests. No CPU-cost study or hidden score is claimed. Public success cannot establish all burst, reconnect-storm or bandwidth behavior. The PDF's completeness levels below 99% and 95% cap hidden marks; stricter team CI and latency/freshness goals are internal criteria.

Before submission: merge this integration PR and review PR #19's cross-component behavior; verify the two-tab demo, review the report with component owners, agree contribution percentages and record the compulsory 5–7 minute video. Passing submission_check.py only verifies its actual checks; it cannot certify these remaining requirements or individual understanding.

Future work / FYP direction

Investigate adaptive semantic scheduling using per-client lag and estimated bandwidth while preserving finals and restoring all revision delivery after impairment. Compare against bounded FIFO using final freshness, normal-reader isolation and CPU/RSS cost. A later FYP could feed a local speech recognizer through the same protocol and study end-to-end delay and usability. ASR remains optional and outside the mandatory deterministic system.